POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern architetturali

Model-View-Controller

Motivazioni

- Implementare un sistema software comporta lavorare su più livelli
- Modello, logica applicativa, CLI, GUI ma anche networking
- Principio della singola responsabilità: ogni componente/classe deve svolgere una ed una sola funzione

Motivazioni

- **Problema:** ogni componente deve avere un compito preciso all'interno del sistema, la gestione delle relazioni tra i vari componenti è complessa
- Minimizzare interdipendenze
- Manutenibilità
- Ciascun livello può potenzialmente essere sviluppato da diversi gruppi di lavoro (e.g., UI sviluppata da designer)

Una soluzione non ideale

```
class Calculator {

        float result = 0;
        String expression = ""
        TextField text;
        Button calculateButton;

        private void parseExpression(){ … }
        private void compute() { … }

        private float power(){ … }
        private float factorial(){}

        public void onClick(Button b){
            if (b == calculateButton){
                compute();
                text.setText(String.valueOf(result));
            }
        }

}
```

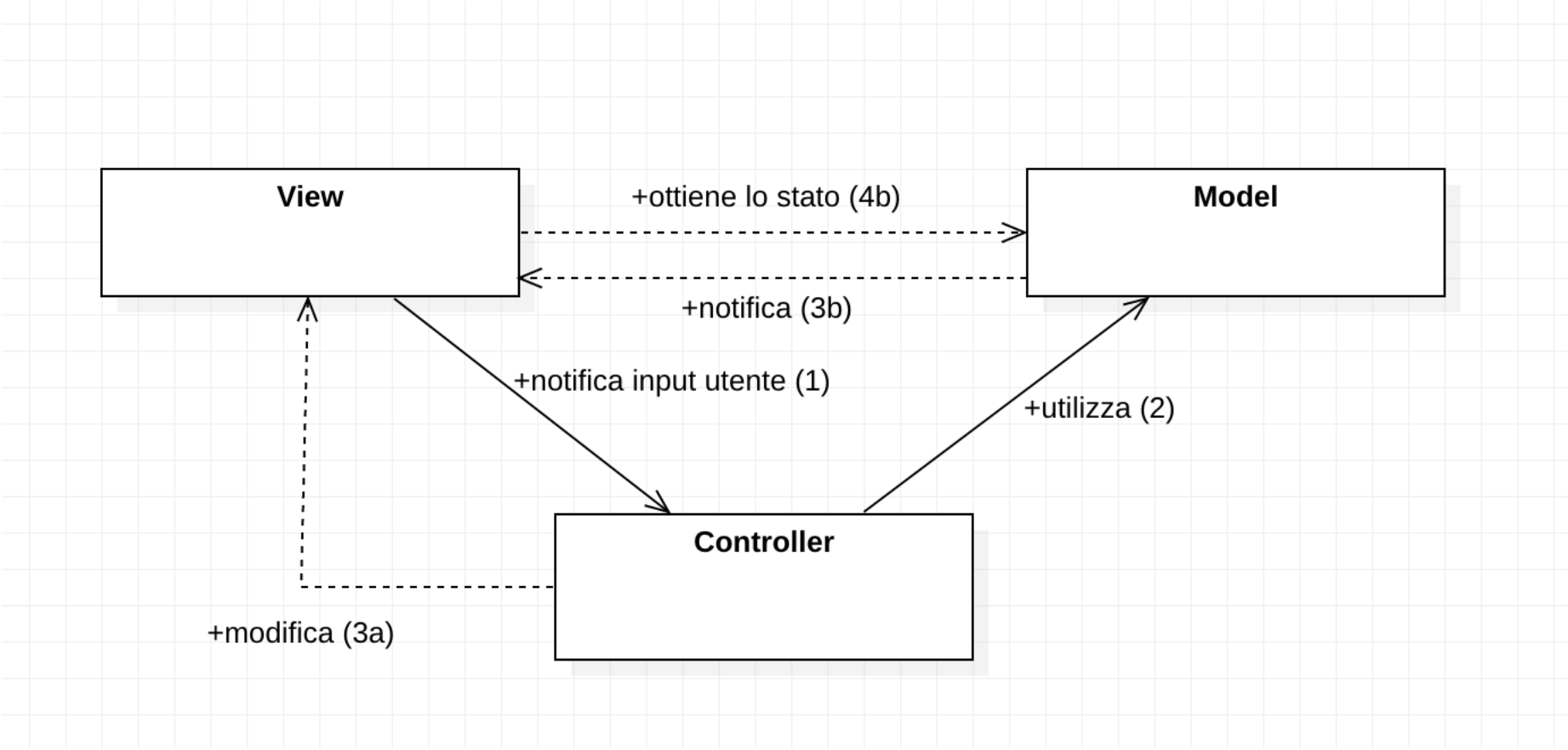
Obbiettivi

- Model-View-Controller (MVC): suddividere lo sviluppo in tre componenti
- Model: contiene lo stato e la logica applicativa e non ha dipendenze con altri componenti
- View*:* contiene la logica di visualizzazione del Model, può avere un riferimento al Model
- Controller: riceve gli input degli utenti inoltrati dalla view, modifica il modello e la view

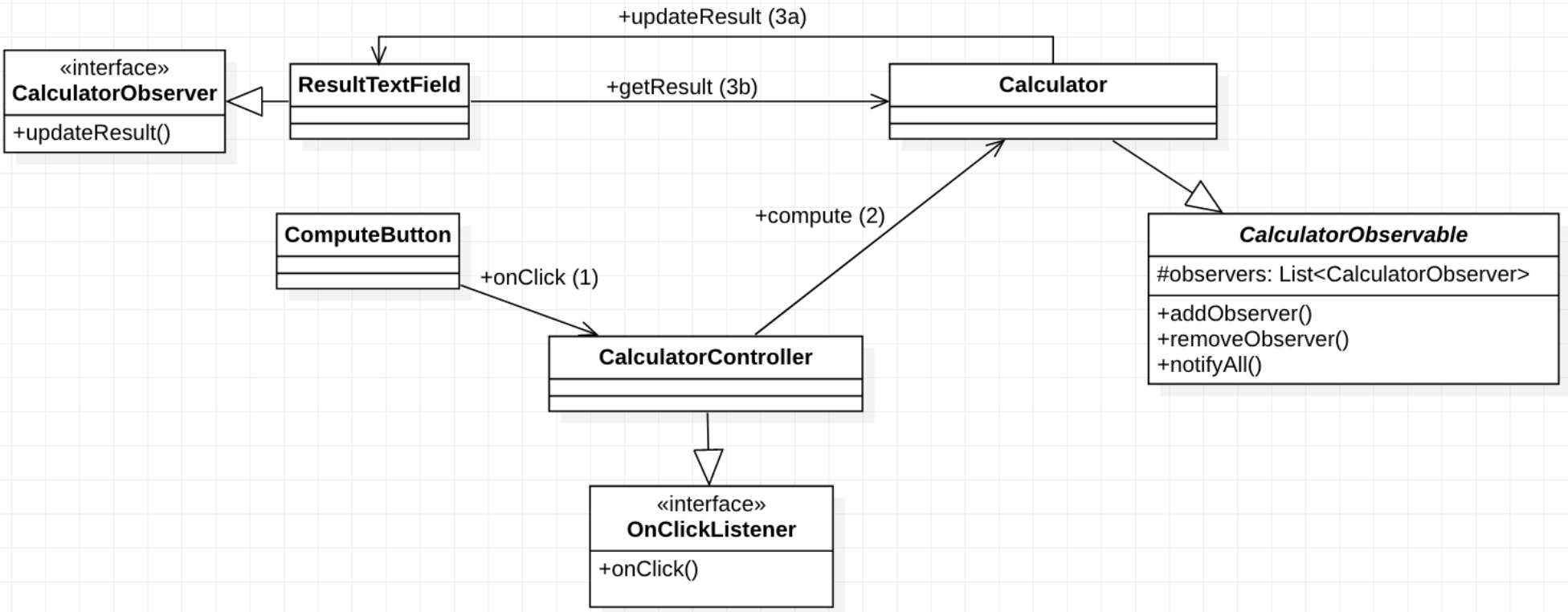
Considerazioni

- Esistono diverse declinazioni di MVC
- Model-View-Presenter, Model-ViewModel-View
- Hanno gli stessi obbiettivi
- Diverse intersezioni

Struttura


View
Model
Controller
+ottiene lo stato (4b)
+notifica (3b)
+notifica input utente (1)
+utilizza (2)
+modifica (3a)

Esempio


+updateResult (3a)
«interface»
CalculatorObserver
+updateResult()
ResultTextField
+getResult (3b)
Calculator
+compute (2)
ComputeButton
+onClick (1)
CalculatorObservable
#observers: List<CalculatorObserver>
+addObserver()
+removeObserver()
+notifyAll()
CalculatorController
«interface»
OnClickListener
+onClick()

Esempio

```
class Calculator extends CalculatorObservable {
      float result = 0;
      String expression = ""

      private void parseExpression(){ … }

      private void compute() {
            …
            notifyAll();
      }

      private float power(){ … }
      private float factorial(){}

      public float getResult() { return result; }
}
```

Esempio

```
class CalculatorController implements OnClickListener {

        ComputeButton button;
        Calculator calculator = new Calculator();

        public void setComputeButton(ComputeButton button){
                this.button = button;
                this.button.addOnClickListener(this);
        }

        public void onClick(){
                calculator.compute();
        }

        …
}
```

Esempio

```
class ComputeButton extends Button {
        // receives users inputs
}

class ResultTextField extends TextField implements CalculatorListener {
        Calculator calc;
        public ResultTextField(Calculator c){
                calc = c;
                c.addObserver(this);

        }

        public void updateResult(){
                setText(String.valueOf(calc.getResult()));
        }
}
```